



Cahier des charges

1. Contexte du projet

. Besoins du client

Suite à une évolution d'activité l'entreprise souhaite pouvoir faire une migration de son application avec des technologies plus d'actualité LiVrai et une entreprise qui effectue des livraisons auprès des particuliers et des entreprises et souhaite aujourd'hui faire en sorte que la gestion des demandes et des livraisons ne soit plus gérée par le service commercial mais par les clients eux-mêmes.

la limite à l'heure actuelle de l'application et le fait qu'elle fonctionne avec une technologie assez peut utiliser à l'heure actuelle et ne permet donc pas une modification de structure rapide

. Objectifs du projet

L'objectif du projet aura pour but de faire cette montée de version en utilisant Java Spring boot côté backend avec un SGBD Hibernate mySQL ou postgres SQL l'idée est d'effectuer cette migration de manière la plus propre possible en incluant également une partie de sécurité. Tout en mettant en place les demandes listées par le client.

- *création de compte par les clients ou par notre service commercial ;*

- *gestion des informations du client ;*
- *commande et suivi des livraisons ;*
- *gestion de la facturation ;*
- *historique des livraisons.*

De plus, l'application devra avoir 3 types d'utilisateurs :

les clients ;

- *notre service commercial qui doit pouvoir gérer certains clients et avoir accès à la facturation ;*
- *notre service Livraisons qui doit pouvoir gérer les livraisons et avoir accès à la facturation.*

. Mesure de la réussite du projet

La réussite du projet sera évaluée à la fois sur des critères fonctionnels et techniques.

Sur le plan fonctionnel, le projet sera considéré comme abouti si les utilisateurs finaux (clients, service commercial, service livraisons) peuvent utiliser l'application de manière autonome pour créer et gérer leurs comptes, passer et suivre des commandes de livraison, accéder à la facturation et consulter l'historique des livraisons. La réduction de la charge de travail du service commercial constituera également un indicateur clé de réussite.

Sur le plan technique, la réussite sera mesurée par la conformité aux choix technologiques validés (Angular, Spring Boot, Docker, MySQL/PostgreSQL), par la mise en place d'une sécurité conforme aux normes (authentification, gestion des accès, respect du RGPD), et par la validation des tests unitaires, d'intégration et de performance. Enfin, la fluidité de la navigation et la maintenabilité du code seront des points essentiels pour garantir la pérennité de la solution.

· Contraintes réglementaires

La future application devra être conforme au **Règlement Général sur la Protection des Données (RGPD)**.

Cela implique notamment :

- que les données personnelles des clients (nom, coordonnées, facturation) soient stockées et traitées de manière sécurisée ;
- que seules les personnes autorisées puissent y accéder, via un système d'authentification et de gestion des droits ;
- que les utilisateurs puissent exercer leurs droits (accès, modification ou suppression de leurs données).

De plus, l'application respectera les bonnes pratiques de sécurité informatique (mots de passe chiffrés, communications sécurisées via HTTPS) et veillera à limiter la durée de conservation des données sensibles aux besoins strictement nécessaires.

· Éléments hors périmètre

Exclure si nécessaire certains aspects pour que le projet reste raisonnable.

Indiquer éventuellement de possibles améliorations qui ne peuvent pas être traitées dans ce projet. ???

2. Description fonctionnelle

· Liste des fonctionnalités

Liste des fonctionnalités du projet. Chaque fonctionnalité doit être décrite et mise en relation avec un besoin exprimé par le client.

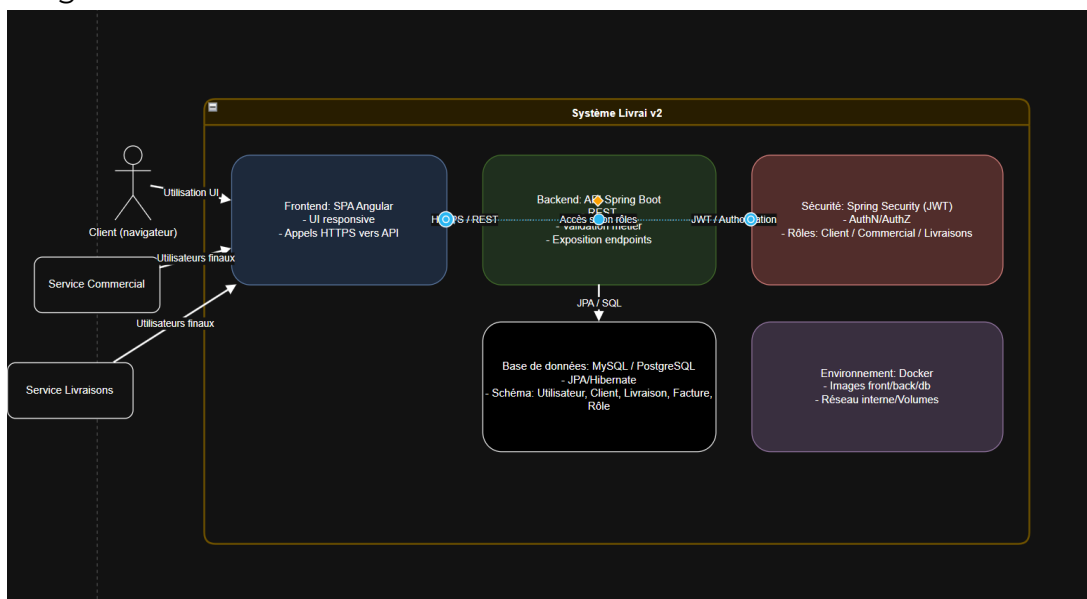
Besoin exprimé par le client	Besoin exprimé par le client
Les clients doivent être autonomes dans la création de compte	Formulaire d'inscription permettant à un client de créer son compte

Gestion des informations du client	Interface de profil client permettant de modifier ses informations
Suivi et commande des livraisons	Tableau de bord client affichant l'état des livraisons et permettant d'en commander
Gestion de la facturation	Module de facturation consultable par le client et le service commercial
Historique des livraisons	Page listant toutes les livraisons passées avec détails

· Fonctionnement de l'application

Présenter le fonctionnement de l'application à l'aide de diagrammes fonctionnels.

Diagramme de conteneurs



1) Ce que voit un utilisateur

- **Client (navigateur)** ouvre l'application web (UI responsive Angular).
 - Il **s'authentifie** (login) → un **JWT** est émis si les identifiants sont valides.
 - Ensuite il effectue ses actions (voir livraisons, créer commande, etc.).
- Les rôles **Client / Commercial / Livraisons** conditionnent ce qu'il peut faire/voir.

2) Frontend — SPA Angular

- Conteneur UI.

- Rend l'interface, gère la navigation et l'état local.
- **Appelle l'API** du backend en **HTTPS/REST** en joignant le **JWT** dans les requêtes.
- Ne contient pas de logique métier critique : elle vit côté domaine.

3) Backend — API Spring Boot (cœur applicatif)

- Conteneur applicatif exposant des **endpoints REST**.
- **Application layer** (au sens DDD) : orchestre les cas d'usage, valide les commandes, publie les DTO de sortie.
- **Domain layer** : règles de gestion (aggregates Utilisateur, Client, Livraison, Facture, Rôle, etc.), invariants, politiques.
- **Ports/Adapters** : contrôleurs REST en entrée ; **repositories JPA** vers la base en sortie.

4) Sécurité — Spring Security (JWT)

- **AuthN/AuthZ** transverses :
 - Authentifie l'utilisateur et délivre le **JWT**.
 - **Autorise** l'accès aux endpoints selon les **rôles** (Client, Commercial, Livraisons).
- Placé logiquement à l'entrée de l'API (filtre/chain Spring Security).

5) Persistance — Base de données (MySQL/PostgreSQL)

- Stocke le **modèle du domaine** (tables pour Utilisateur, Client, Livraison, Facture, Rôle).
- Accès via **JPA/Hibernate** (SQL généré), respectant les **Agrégats** et **Repositories DDD**.

6) Environnement — Docker

- Chaque conteneur (frontend, backend, db) est packagé en image Docker.
- Réseau interne + volumes assurent la communication et la persistance.
- Simplifie le déploiement et l'isolation.

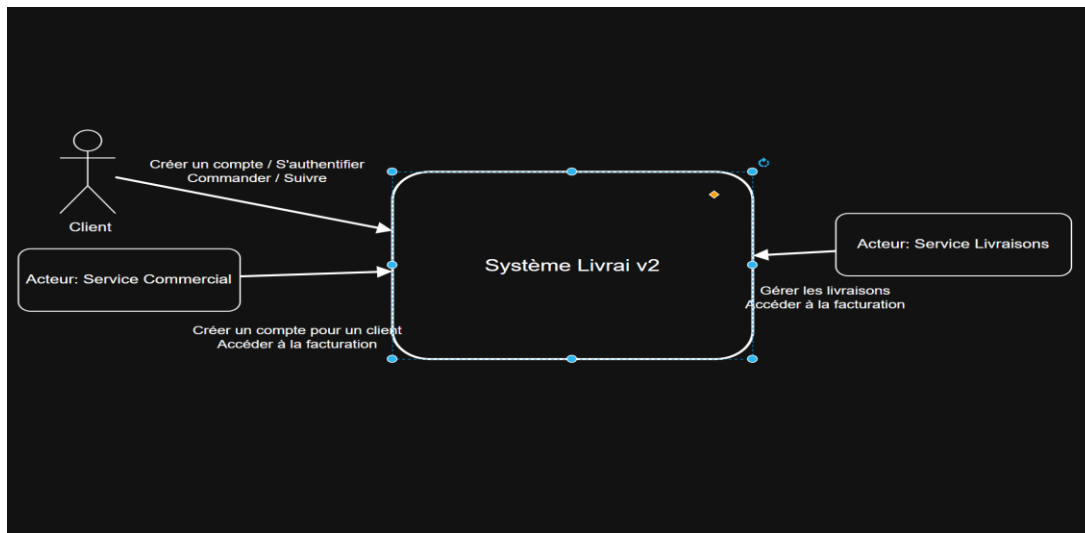
7) Flux principal (de bout en bout)

1. Le navigateur envoie une **requête HTTPS** à l'API via l'UI Angular.
2. **Spring Security** vérifie le **JWT** et les **rôles**.
3. **Application service** appelle le **domaine** (agregats, règles).
4. Le domaine lit/écrit via **Repositories JPA** dans la **BD**.
5. L'API renvoie un **DTO** → l'UI met à jour l'écran.

8) Lecture DDD du diagramme

- **Bounded Contexts** implicites : Comptes (auth/roles), Clients, Livraisons, Facturation.
- **Agrégats** candidats : Utilisateur (avec Rôle), Client, Livraison, Facture.
- **Repositories** : un par agrégat ; implémentés via JPA.
- **Services d'application** : encapsulent les cas d'usage (ex. Créer une livraison, Valider une facture).
- **Sécurité** = préoccupation transversale, non métier, appliquée au bord du système

Diagramme de contexte



Ce que couvre le système

- **Système Livrai v2** : cœur applicatif unique qui gère l'authentification, les commandes, le suivi et la facturation, ainsi que la gestion opérationnelle des livraisons.

Acteurs externes et interactions

- **Client**
 - Créer un compte / S'authentifier : demande d'inscription puis connexion.
 - Commander / Suivre : crée une commande de livraison et consulte son état.
- **Service Commercial**
 - Créer un compte pour un client : on-boarding quand le client ne peut pas s'auto-inscrire.
 - Accéder à la facturation : consulte/édite les documents de facturation liés aux clients.
- **Service Livraisons**
 - Gérer les livraisons : planifie, assigne, met à jour les statuts (prise en charge, en cours, livré...).
 - Accéder à la facturation : valide les éléments nécessaires (ex. preuve de livraison) pour la facturation.

Frontières (ce qui est dedans/dehors)

- **Dedans** : règles métiers, API d'accès, persistance des clients/commandes/livraisons/factures.
- **Dehors** : les utilisateurs (clients et équipes internes) et leurs outils ; ils consomment le système via des interfaces exposées.

En bref : trois **acteurs** interagissent avec un **système central**. Le client passe commande et suit; le commercial crée des comptes et gère la facturation; le service livraisons opère les courses et alimente la facturation.

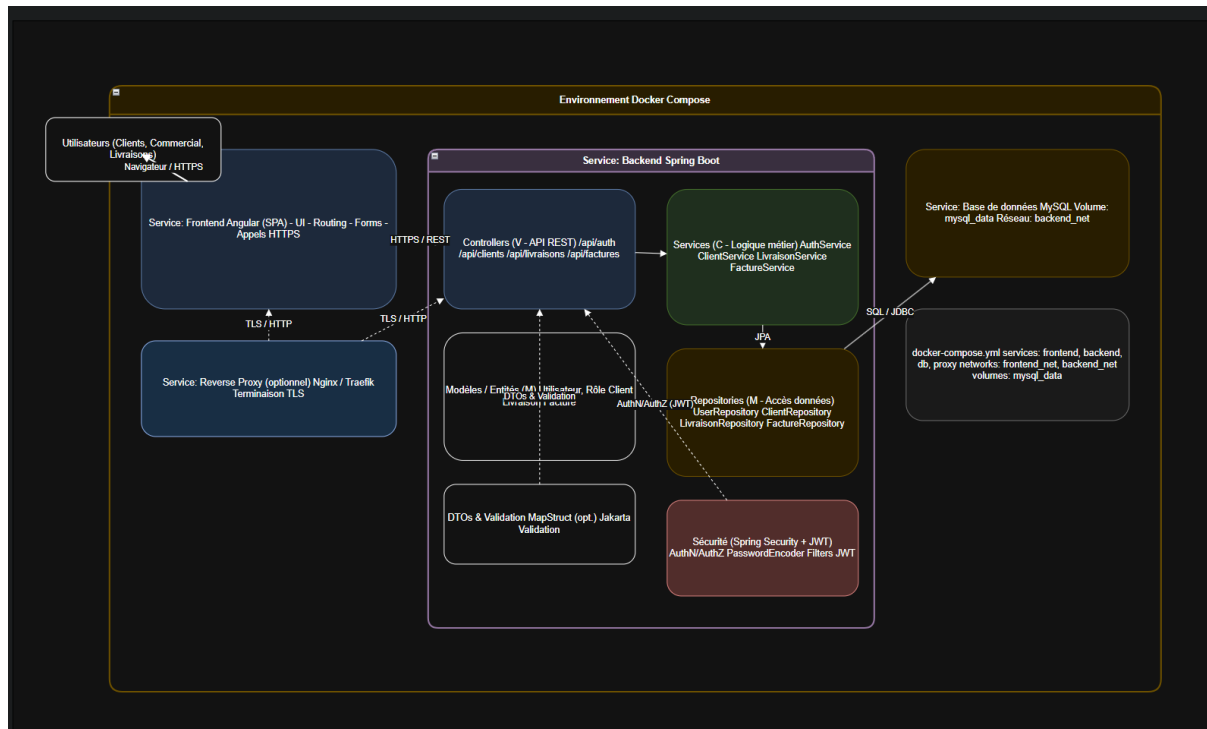
• Impact Social

Présenter brièvement l'impact sur l'environnement de l'application et l'utilisabilité de l'application pour les publics en situation de handicap.

3. Description technique

Présenter l'ensemble des technologies utilisées en les justifiant.

Inclure un diagramme de l'architecture de l'application.



Service : Frontend Angular (SPA)

UI, routing, formulaires. Le navigateur des **utilisateurs** (Clients, Commercial, Livraison) accède au front en **HTTPS**.

Le front consomme l'API en **REST/JSON** via TLS/HTTPS (directement ou via un reverse-proxy).

Reverse proxy (optionnel) : Nginx / Traefik

Terminaison TLS, routage des requêtes /api/* vers le backend, et du reste vers l'app Angular. Utile en prod, facultatif en dev.

Backend

• Service : Backend Spring Boot

Découpé en couches internes :

- **Contrôleurs (V – API REST)** : exposition des endpoints (/api/auth, /api/clients, /api/livraisons, /api/factures).
- **Services (C – Logique métier)** : orchestration des règles, sécurité applicative, validations.
- **Repositories (M – Accès données)** : Spring Data JPA pour les agrégats (User/Client/Commande/Livraison/Facture).

- **Modèles & DTOs** : Entités JPA pour la persistance, **DTOs** pour les échanges API, **MapStruct** (optionnel) pour le mapping, **Jakarta Validation** pour les contraintes.
- **Sécurité : Spring Security + JWT** (filtres d'authentification/autorisation, PasswordEncoder, gestion des rôles CLIENT/COMMERCIAL/LIVRAISON). Tous les appels API passent par les filtres JWT.

Données

- **Service : Base MySQL**

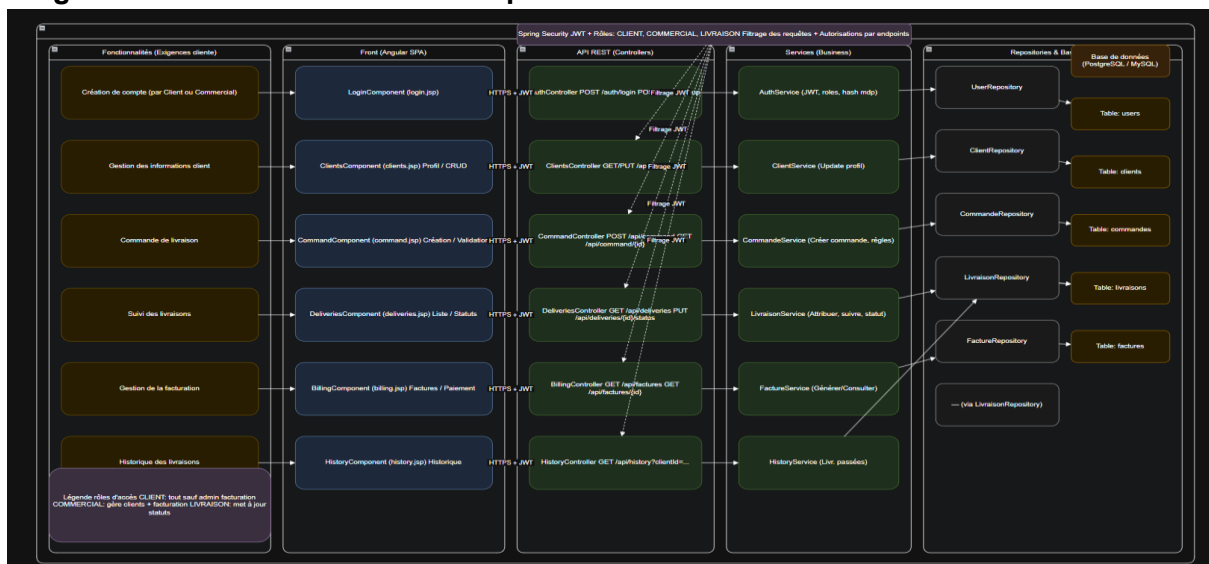
Persistance des tables de domaine

(utilisateurs/clients/commandes/livraisons/factures). Les **Repositories** accèdent à la DB via **JPA** → **JDBC/SQL**. Les données sont stockées sur un **volume** persistant.

Flux principaux

1. Le navigateur charge la SPA Angular en **HTTPS**.
2. La SPA appelle l'API (/api/...) en **REST/JSON** ; le **JWT** est envoyé dans l'en-tête Authorization.
3. Le backend passe par **Security (JWT)** → **Controller** → **Service** → **Repository** → **DB**, puis renvoie la réponse JSON.

Diagramme de l'architecture technique



Ce schéma montre, de gauche à droite, la **traçabilité complète** d'une fonctionnalité métier jusqu'à la base de données :

- **Fonctionnalités** : les besoins client (création de compte, gestion client, commande, suivi, facturation, historique) servent de point d'entrée.
- **Front (Angular SPA)** : chaque fonctionnalité est portée par un **composant Angular** nommé comme l'existant (ex. Login, Clients, Deliveries), garantissant la cohérence de vocabulaire.
- **API REST (Spring Boot – Controllers)** : les composants front appellent des **endpoints REST** (ex. /api/auth, /api/clients, /api/deliveries, /api/factures) en **HTTPS**.
- **Services (Business)** : la logique métier est centralisée dans des services dédiés (AuthService, ClientService, CommandService, LivraisonService, FactureService).
- **Repositories (JPA)** : l'accès aux données passe par **Spring Data JPA** (UserRepository, ClientRepository, LivraisonRepository, FactureRepository), assurant une séparation nette des responsabilités.

- **Base de données SQL** : persistance des agrégats principaux (users, clients, commandes, livraisons, factures).
- **Sécurité** : **Spring Security + JWT** filtre toutes les requêtes ; les **rôles** (CLIENT, COMMERCIAL, LIVRAISON) portent l'**autorisation par endpoint**.
- **Objectif du design** : architecture lisible, **maintenable** et **évolutive**, où chaque besoin est **aligné** sur un flux **Front** → **API** → **Service** → **Repository** → **Table**.